

No- B_k 系数拆分自举因式分解 与 subKey Bootstrapping 叶子优化

面向 CKKS Bootstrapping 的
coefficient-split core factorization

组会汇报

2026 年 5 月 8 日

CKKS 编码与问题背景

从明文环开始

CKKS 的明文不是一开始就写成“向量”，而是先放在一个多项式环里。

消息层的复数环

$$\mathcal{R}_d = \mathbb{C}[X]/(X^d + 1).$$

一个消息多项式写作

$$P(X) = a_0 + a_1X + \cdots + a_{d-1}X^{d-1}.$$

从明文环开始

CKKS 的明文不是一开始就写成“向量”，而是先放在一个多项式环里。

消息层的复数环

$$\mathcal{R}_d = \mathbb{C}[X]/(X^d + 1).$$

一个消息多项式写作

$$P(X) = a_0 + a_1X + \cdots + a_{d-1}X^{d-1}.$$

密文计算使用的模环

$$\mathcal{R}_{q,d} = \mathbb{Z}_q[X]/(X^d + 1).$$

CKKS 编码会把复数消息经过 scale Δ 放入这个模环中：

$$\text{Encode}_{\Delta,d}(P) \approx \Delta \cdot P \pmod{q}.$$

后面的所有“slot 语义”和“系数拆分”都发生在这个多项式环结构之上。

典范嵌入：CKKS slots 从哪里来？

令

$$\Omega_d = \{\omega \in \mathbb{C} : \omega^d + 1 = 0\}.$$

典范嵌入 (canonical embedding) 把多项式评价到所有根上：

$$\text{can}_d(P) = (P(\omega))_{\omega \in \Omega_d}.$$

典范嵌入：CKKS slots 从哪里来？

令

$$\Omega_d = \{\omega \in \mathbb{C} : \omega^d + 1 = 0\}.$$

典范嵌入 (canonical embedding) 把多项式评价到所有根上：

$$\text{can}_d(P) = (P(\omega))_{\omega \in \Omega_d}.$$

系数坐标

$$\text{coeff}_d(P) = (a_0, \dots, a_{d-1}).$$

这是多项式本身的系数。

槽坐标

$$\text{can}_d(P) = (P(\omega_0), \dots, P(\omega_{d-1})).$$

这是 CKKS SIMD slots 的数学来源。

典范嵌入：CKKS slots 从哪里来？

令

$$\Omega_d = \{\omega \in \mathbb{C} : \omega^d + 1 = 0\}.$$

典范嵌入 (canonical embedding) 把多项式评价到所有根上：

$$\text{can}_d(P) = (P(\omega))_{\omega \in \Omega_d}.$$

系数坐标

$$\text{coeff}_d(P) = (a_0, \dots, a_{d-1}).$$

这是多项式本身的系数。

槽坐标

$$\text{can}_d(P) = (P(\omega_0), \dots, P(\omega_{d-1})).$$

这是 CKKS SIMD slots 的数学来源。

需要区分

一个密文加密的是同一个多项式 $P(X)$ ，只是我们可以用“系数坐标”或“槽坐标”两种方式描述它。

一个小例子：系数坐标和槽坐标不是同一回事

取 $d = 2$, 则

$$\mathcal{R}_2 = \mathbb{C}[X]/(X^2 + 1), \quad \Omega_2 = \{i, -i\}.$$

若

$$P(X) = a_0 + a_1X,$$

那么

$$\text{coeff}_2(P) = (a_0, a_1),$$

但

$$\text{can}_2(P) = (P(i), P(-i)) = (a_0 + ia_1, a_0 - ia_1).$$

一个小例子：系数坐标和槽坐标不是同一回事

取 $d = 2$, 则

$$\mathcal{R}_2 = \mathbb{C}[X]/(X^2 + 1), \quad \Omega_2 = \{i, -i\}.$$

若

$$P(X) = a_0 + a_1X,$$

那么

$$\text{coeff}_2(P) = (a_0, a_1),$$

但

$$\text{can}_2(P) = (P(i), P(-i)) = (a_0 + ia_1, a_0 - ia_1).$$

后面会反复使用的观点

slots 不是“多项式的系数”。它们是同一个多项式在根上的取值。C2S/S2C 的作用，本质上是在这两套坐标之间做线性变换。

密文语义：我们如何说一个密文加密了什么？

本文使用下面的记号：

$$\mathbf{ct} \models_{d,\Delta,s} P(X).$$

它表示：密文 $\mathbf{ct}$ 在环维度 d 、scale Δ 、密钥 s 下加密消息多项式 $P(X)$ 。

密文语义：我们如何说一个密文加密了什么？

本文使用下面的记号：

$$\mathbf{ct} \models_{d,\Delta,s} P(X).$$

它表示：密文 $\mathbf{ct}$ 在环维度 d 、scale Δ 、密钥 s 下加密消息多项式 $P(X)$ 。
更具体地说：

$$\text{Dec}_s(\mathbf{ct}) = \text{Encode}_{\Delta,d}(P) + e \quad \text{in } \mathcal{R}_{q,d},$$

其中 e 是 CKKS 允许的小误差。

密文语义：我们如何说一个密文加密了什么？

本文使用下面的记号：

$$\mathbf{ct} \models_{d,\Delta,s} P(X).$$

它表示：密文 $\mathbf{ct}$ 在环维度 d 、scale Δ 、密钥 s 下加密消息多项式 $P(X)$ 。
更具体地说：

$$\text{Dec}_s(\mathbf{ct}) = \text{Encode}_{\Delta,d}(P) + e \quad \text{in } \mathcal{R}_{q,d},$$

其中 e 是 CKKS 允许的小误差。

槽语义

$$\mathbf{ct} \models_d^{\text{slot}} z \iff \mathbf{ct} \models_d P(X) \text{ 且 } z = \text{can}_d(P).$$

这个记号很重要：后面所有算法仍然作用在密文上， $\models$ 只是记录它诱导出的消息变换。

C2S 与 S2C: Bootstrapping 前必须理解的两个线性变换

在理想消息层:

$$\text{C2S}_d = \text{coeff}_d \circ \text{can}_d^{-1}, \quad \text{S2C}_d = \text{can}_d \circ \text{coeff}_d^{-1}.$$

C2S 与 S2C: Bootstrapping 前必须理解的两个线性变换

在理想消息层:

$$\text{C2S}_d = \text{coeff}_d \circ \text{can}_d^{-1}, \quad \text{S2C}_d = \text{can}_d \circ \text{coeff}_d^{-1}.$$

C2S 的语义

如果

$$\mathbf{ct} \models_d P(X),$$

那么

$$\text{C2S}_d(\mathbf{ct}) \models_d^{\text{slot}} \text{coeff}_d(P).$$

C2S 与 S2C: Bootstrapping 前必须理解的两个线性变换

在理想消息层:

$$\text{C2S}_d = \text{coeff}_d \circ \text{can}_d^{-1}, \quad \text{S2C}_d = \text{can}_d \circ \text{coeff}_d^{-1}.$$

C2S 的语义

如果

$$\mathbf{ct} \models_d P(X),$$

那么

$$\text{C2S}_d(\mathbf{ct}) \models_d^{\text{slot}} \text{coeff}_d(P).$$

关键点

C2S 之后, 密文的 slots 里装的是 $P(X)$ 的系数向量。这类 slots 后面称为系数打包槽 (**coefficient-packed slots**)。

标准 CKKS 自举管线

$$\underbrace{\text{ScaleDown}_N \rightarrow \text{ModRaise}_N}_{\text{预处理}} \rightarrow \underbrace{\text{C2S}_N \rightarrow \text{EvalMod}_N \rightarrow \text{S2C}_N}_{\text{自举核心}}$$

标准 CKKS 自举管线

$$\underbrace{\text{ScaleDown}_N \rightarrow \text{ModRaise}_N}_{\text{预处理}} \rightarrow \underbrace{\text{C2S}_N \rightarrow \text{EvalMod}_N \rightarrow \text{S2C}_N}_{\text{自举核心}}$$

- C2S: CoeffToSlot, 系数到槽变换
- EvalMod: EvalMod, 逐 slot 的近似模约化
- S2C: SlotToCoeff, 槽到系数变换

标准 CKKS 自举管线

$$\underbrace{\text{ScaleDown}_N \rightarrow \text{ModRaise}_N}_{\text{预处理}} \rightarrow \underbrace{\text{C2S}_N \rightarrow \text{EvalMod}_N \rightarrow \text{S2C}_N}_{\text{自举核心}}$$

- C2S: CoeffToSlot, 系数到槽变换
- EvalMod: EvalMod, 逐 slot 的近似模约化
- S2C: SlotToCoeff, 槽到系数变换

代价来源

大环维度 N 上的 NTT、旋转、密钥切换。

粗略模型: $T_{\text{op}}(N, Q) \approx C \cdot N \log N \cdot \#\text{primes}(Q)$

前面的讨论说明，CKKS Bootstrapping 的核心是一个坐标变换夹着逐坐标非线性函数的结构：

$$C2S \rightarrow \text{EvalMod} \rightarrow S2C.$$

前面的讨论说明，CKKS Bootstrapping 的核心是一个坐标变换夹着逐坐标非线性函数的结构：

$$\text{C2S} \rightarrow \text{EvalMod} \rightarrow \text{S2C}.$$

现在考虑 $N = kn$ 。如果能把大环上的一个 Bootstrapping 拆成 k 个叶子环上的 Bootstrapping，那么主要计算就可以在更小的环 n 上进行：

$$\mathbf{ct}_N \xrightarrow{\text{Split}} (\mathbf{ct}_{n,0}, \dots, \mathbf{ct}_{n,k-1}) \xrightarrow{\text{leaf Bootstrapping}} \dots \xrightarrow{\text{Merge}} \mathbf{ct}'_N.$$

前面的讨论说明，CKKS Bootstrapping 的核心是一个坐标变换夹着逐坐标非线性函数的结构：

$$\text{C2S} \rightarrow \text{EvalMod} \rightarrow \text{S2C}.$$

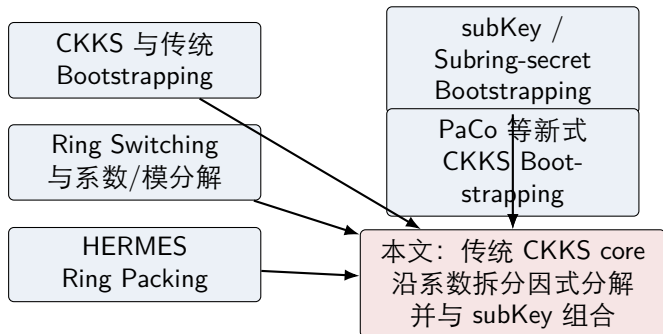
现在考虑 $N = kn$ 。如果能把大环上的一个 Bootstrapping 拆成 k 个叶子环上的 Bootstrapping，那么主要计算就可以在更小的环 n 上进行：

$$\mathbf{ct}_N \xrightarrow{\text{Split}} (\mathbf{ct}_{n,0}, \dots, \mathbf{ct}_{n,k-1}) \xrightarrow{\text{leaf Bootstrapping}} \dots \xrightarrow{\text{Merge}} \mathbf{ct}'_N.$$

直接挑战

- 系数拆分 (Split_k) 不保持原始 canonical slots
- 如果恢复 slot 需要额外的 B_k 变换 (k 个叶子间的线性变换)
- 本文证明：对 CKKS Bootstrapping core 来说，恢复原始 slots 不是必要条件

Related Work 总览：我们站在哪几条线上？



CKKS 做什么？

CKKS 用于近似实数/复数计算。用户看到的是复数向量；数学底层是 cyclotomic ring 中的多项式。

Related Work 1: CKKS 与传统 Bootstrapping

CKKS 做什么？

CKKS 用于近似实数/复数计算。用户看到的是复数向量；数学底层是 cyclotomic ring 中的多项式。

传统 CKKS Bootstrapping 的核心

$\text{CoeffToSlot} \rightarrow \text{EvalMod} \rightarrow \text{SlotToCoeff}$.

现有工作通常优化：

- 线性变换 C2S/S2C 的分解与旋转数量；
- EvalMod 的多项式近似；
- 模数链、scale、精度和安全参数。

Related Work 1: CKKS 与传统 Bootstrapping

CKKS 做什么？

CKKS 用于近似实数/复数计算。用户看到的是复数向量；数学底层是 cyclotomic ring 中的多项式。

传统 CKKS Bootstrapping 的核心

$\text{CoeffToSlot} \rightarrow \text{EvalMod} \rightarrow \text{SlotToCoeff}$.

现有工作通常优化：

- 线性变换 C2S/S2C 的分解与旋转数量；
- EvalMod 的多项式近似；
- 模数链、scale、精度和安全参数。

本文与它们的区别

本文不重新设计 CKKS，也不替换 EvalMod。

本文问的是：传统 **C2S** $\rightarrow$ **EvalMod** $\rightarrow$ **S2C core** 本身能否沿系数拆分分解？

经典分解

当 $N = kn$ 时:

$$P(X) = \sum_{r=0}^{k-1} X^r P_r(X^k), \quad P_r(Y) \in \mathbb{C}[Y]/(Y^n + 1).$$

这就是把大环多项式按系数下标模 k 分组。

经典分解

当 $N = kn$ 时:

$$P(X) = \sum_{r=0}^{k-1} X^r P_r(X^k), \quad P_r(Y) \in \mathbb{C}[Y]/(Y^n + 1).$$

这就是把大环多项式按系数下标模 k 分组。

- 这类分解和 ring switching 不是本文新发明。
- 密文层面的 ring switching 需要 key switching, 会引入噪声。
- 本文新点不是“可以拆多项式”, 而是:

本文新问题

这种 coefficient/module decomposition 是否与 CKKS Bootstrapping 的 sandwich 结构兼容?

HERMES 的任务

HERMES 使用 ring switching 和 MLWE 中间表示，把许多 LWE/MLWE 格式密文打包成一个 RLWE/CKKS 密文，用于 transciphering 和后续 SIMD 计算。

Related Work 3: HERMES 与 Ring Packing

HERMES 的任务

HERMES 使用 ring switching 和 MLWE 中间表示，把许多 LWE/MLWE 格式密文打包成一个 RLWE/CKKS 密文，用于 transciphering 和后续 SIMD 计算。

HERMES

- 输入：很多 LWE/MLWE 密文
- 目标：打包成 RLWE/CKKS
- 关键词：ring packing, transciphering

本文

- 输入：已经是 CKKS/RLWE 密文
- 目标：分解它自己的 Bootstrapping core
- 关键词：core factorization

Related Work 3: HERMES 与 Ring Packing

HERMES 的任务

HERMES 使用 ring switching 和 MLWE 中间表示，把许多 LWE/MLWE 格式密文打包成一个 RLWE/CKKS 密文，用于 transciphering 和后续 SIMD 计算。

HERMES

- 输入：很多 LWE/MLWE 密文
- 目标：打包成 RLWE/CKKS
- 关键词：ring packing, transciphering

本文

- 输入：已经是 CKKS/RLWE 密文
- 目标：分解它自己的 Bootstrapping core
- 关键词：core factorization

区别

HERMES 解决的是“把许多小密文打包成大密文”；本文解决的是“一个大密文的自举能否拆成多个叶子自举”。

subKey Bootstrapping 的思想

在 Bootstrapping 过程中切换到子环密钥结构，使若干步骤更便宜：

- dense secret 的有效 Hamming weight 变小；
- EvalMod 或 digit removal 的近似区间缩小；
- C2S/S2C 中的 key switching 可以利用子环结构。

subKey Bootstrapping 的思想

在 Bootstrapping 过程中切换到子环密钥结构，使若干步骤更便宜：

- dense secret 的有效 Hamming weight 变小；
- EvalMod 或 digit removal 的近似区间缩小；
- C2S/S2C 中的 key switching 可以利用子环结构。

subKey 方法

- secret-side optimization
- 仍围绕一个密文做自举
- 改变的是密钥结构

本文 $\text{no-}B_k$

- ciphertext/message-side decomposition
- 把密文语义拆到多个 leaf
- 改变的是执行粒度

Related Work 4: subKey / subring-secret Bootstrapping

subKey Bootstrapping 的思想

在 Bootstrapping 过程中切换到子环密钥结构，使若干步骤更便宜：

- dense secret 的有效 Hamming weight 变小；
- EvalMod 或 digit removal 的近似区间缩小；
- C2S/S2C 中的 key switching 可以利用子环结构。

subKey 方法

- secret-side optimization
- 仍围绕一个密文做自举
- 改变的是密钥结构

本文 $\text{no-}B_k$

- ciphertext/message-side decomposition
- 把密文语义拆到多个 leaf
- 改变的是执行粒度

为什么能组合？

$\text{no-}B_k$ 是外层因式分解；subKey 是叶子内部的自举引擎。两者不是互斥，而是可以叠加。

Related Work 5: PaCo 与新式 CKKS Bootstrapping

PaCo 的方向

PaCo 通过 partial CoeffToSlot、blind rotations、circle-group modular additions 和 alternative ring structures 改写 CKKS Bootstrapping 逻辑。

Related Work 5: PaCo 与新式 CKKS Bootstrapping

PaCo 的方向

PaCo 通过 partial CoeffToSlot、blind rotations、circle-group modular additions 和 alternative ring structures 改写 CKKS Bootstrapping 逻辑。

本文与 PaCo 正交

- 不重新表述 CKKS decryption equation;
- 不用 blind rotation 作为主机制;
- 不替换传统 EvalMod;
- 只改变传统 core 的执行粒度。

Related Work 5: PaCo 与新式 CKKS Bootstrapping

PaCo 的方向

PaCo 通过 partial CoeffToSlot、blind rotations、circle-group modular additions 和 alternative ring structures 改写 CKKS Bootstrapping 逻辑。

本文与 PaCo 正交

- 不重新表述 CKKS decryption equation;
- 不用 blind rotation 作为主机制;
- 不替换传统 EvalMod;
- 只改变传统 core 的执行粒度。

传统 core + 系数拆分因式分解

本文处理的对象

本文不改变 CKKS 的编码方式，也不替换传统 EvalMod。研究对象是传统 Bootstrapping core

$$\text{C2S} \rightarrow \text{EvalMod} \rightarrow \text{S2C}$$

在 coefficient split 下是否保持同一个理想消息变换。

本文处理的对象

本文不改变 CKKS 的编码方式，也不替换传统 EvalMod。研究对象是传统 Bootstrapping core

$$\text{C2S} \rightarrow \text{EvalMod} \rightarrow \text{S2C}$$

在 coefficient split 下是否保持同一个理想消息变换。

- ① 语义层面：C2S 后的 slots 是系数打包槽 (coefficient-packed slots)，而不是原始 canonical slots。
- ② 理论层面：抽象出 split-compatible sandwich theorem，并用三个兼容性引理实例化到 CKKS。
- ③ 实现层面：实现 split-first leaf preprocessing，并在叶子上替换为 subKey Bootstrapping 做实验验证。

系数拆分的语义基础

符号约定 (I)

- 环: $\mathcal{R}_d = \mathbb{C}[X]/(X^d + 1)$, $\mathcal{R}_{q,d} = \mathbb{Z}_q[X]/(X^d + 1)$
- 语义判断: $\mathbf{ct} \models_d P(X)$ (密文加密 $P(X)$)
- 典范嵌入: $\text{can}_d : \mathcal{R}_d \rightarrow \text{Can}_d$, $P \mapsto (P(\omega))_{\omega^d+1=0}$
- 系数坐标: $\text{coeff}_d : \mathcal{R}_d \rightarrow \text{Coeff}_d$, $P(X) = \sum a_i X^i \mapsto (a_0, \dots, a_{d-1})$

符号约定 (I)

- 环: $\mathcal{R}_d = \mathbb{C}[X]/(X^d + 1)$, $\mathcal{R}_{q,d} = \mathbb{Z}_q[X]/(X^d + 1)$
- 语义判断: $\mathbf{ct} \models_d P(X)$ (密文加密 $P(X)$)
- 典范嵌入: $\text{can}_d : \mathcal{R}_d \rightarrow \text{Can}_d$, $P \mapsto (P(\omega))_{\omega^d+1=0}$
- 系数坐标: $\text{coeff}_d : \mathcal{R}_d \rightarrow \text{Coeff}_d$, $P(X) = \sum a_i X^i \mapsto (a_0, \dots, a_{d-1})$

Slot 语义

$$\mathbf{ct} \models_d^{\text{slot}} z \iff \mathbf{ct} \models_d P(X) \text{ 且 } z = \text{can}_d(P(X))$$

Slot 是加密消息多项式在典范嵌入下的坐标——不是独立的明文对象。

符号约定 (II): C2S / S2C

坐标映射 (理想消息层)

$$\text{C2S}_d = \text{coeff}_d \circ \text{can}_d^{-1}$$

$$\text{S2C}_d = \text{can}_d \circ \text{coeff}_d^{-1}$$

符号约定 (II): C2S / S2C

坐标映射 (理想消息层)

$$\text{C2S}_d = \text{coeff}_d \circ \text{can}_d^{-1}$$

$$\text{S2C}_d = \text{can}_d \circ \text{coeff}_d^{-1}$$

语义效果

$$\mathbf{ct} \models_d P(X) \implies \text{C2S}_d(\mathbf{ct}) \models_d^{\text{slot}} \text{coeff}_d(P(X))$$

$$\mathbf{ct}' \models_d^{\text{slot}} y \implies \text{S2C}_d(\mathbf{ct}') \models_d \text{coeff}_d^{-1}(y)$$

符号约定 (II): C2S / S2C

坐标映射 (理想消息层)

$$\text{C2S}_d = \text{coeff}_d \circ \text{can}_d^{-1}$$

$$\text{S2C}_d = \text{can}_d \circ \text{coeff}_d^{-1}$$

语义效果

$$\text{ct} \models_d P(X) \implies \text{C2S}_d(\text{ct}) \models_d^{\text{slot}} \text{coeff}_d(P(X))$$

$$\text{ct}' \models_d^{\text{slot}} y \implies \text{S2C}_d(\text{ct}') \models_d \text{coeff}_d^{-1}(y)$$

定义 (Coefficient-Packed Slots)

ct' 携带 $P(X)$ 的 **coefficient-packed slots**, 当且仅当

$$\text{ct}' \models_d^{\text{slot}} \text{coeff}_d(P(X))$$

这意味着 C2S 之后, slot 里装的是 **系数坐标**, 不是原始 slot 值!

分量形式

$$\text{EvalMod}_d = f^{\oplus d}, \quad f^{\oplus d}(x_0, \dots, x_{d-1}) = (f(x_0), \dots, f(x_{d-1}))$$

分量形式

$$\text{EvalMod}_d = f^{\oplus d}, \quad f^{\oplus d}(x_0, \dots, x_{d-1}) = (f(x_0), \dots, f(x_{d-1}))$$

其中 $f : \mathbb{C} \rightarrow \mathbb{C}$ 是公开的标量模约化函数。

多项式近似: $p(t) = \sum_{\ell=0}^m c_\ell t^\ell$, 同态计算 $p(Q(X))$ 。

分量形式

$$\text{EvalMod}_d = f^{\oplus d}, \quad f^{\oplus d}(x_0, \dots, x_{d-1}) = (f(x_0), \dots, f(x_{d-1}))$$

其中 $f : \mathbb{C} \rightarrow \mathbb{C}$ 是公开的标量模约化函数。

多项式近似: $p(t) = \sum_{\ell=0}^m c_\ell t^\ell$, 同态计算 $p(Q(X))$ 。

关键观察

EvalMod 对 **C2S** 之后的 **slots** 分量作用。

这些 slots 装载的是 **系数坐标**, 不是原始 big-ring slots。

⇒ 系数拆分的“不保 slot”不影响自举正确性。

设 $N = kn$, $Y \mapsto X^k$ 给出 $\mathcal{R}_n \hookrightarrow \mathcal{R}_N$ 。

系数分解

$$P(X) = \sum_{r=0}^{k-1} X^r P_r(X^k), \quad P_r(Y) \in \mathcal{R}_n$$

其中 $P_r(Y) = \sum_{j=0}^{n-1} a_{r+jk} Y^j$

子环分解

设 $N = kn$, $Y \mapsto X^k$ 给出 $\mathcal{R}_n \hookrightarrow \mathcal{R}_N$ 。

系数分解

$$P(X) = \sum_{r=0}^{k-1} X^r P_r(X^k), \quad P_r(Y) \in \mathcal{R}_n$$

其中 $P_r(Y) = \sum_{j=0}^{n-1} a_{r+jk} Y^j$

余类分组算子 Π_k

$$S_r p = (a_r, a_{r+k}, \dots, a_{r+(n-1)k}) \in \text{Coeff}_n$$

$$\boxed{\Pi_k : \text{Coeff}_N \rightarrow \bigoplus_{r=0}^{k-1} \text{Coeff}_n}, \quad \Pi_k p = (S_0 p, \dots, S_{k-1} p)$$

直观：把长 N 的系数向量按「模 k 余类」分成 k 个长为 n 的子向量。

Split_k 的语义规格

$$\mathbf{ct}_N \models_N P(X) \implies \text{Split}_k(\mathbf{ct}_N) = (\mathbf{ct}_{n,0}, \dots, \mathbf{ct}_{n,k-1})$$

且 $\mathbf{ct}_{n,r} \models_n P_r(Y)$ 。

系数拆分 = 密文拆分

Split_k 的语义规格

$$\mathbf{ct}_N \models_N P(X) \implies \text{Split}_k(\mathbf{ct}_N) = (\mathbf{ct}_{n,0}, \dots, \mathbf{ct}_{n,k-1})$$

且 $\mathbf{ct}_{n,r} \models_n P_r(Y)$ 。

Merge_k 的语义规格（逆操作）

$$\mathbf{ct}_{n,r} \models_n P_r(Y) \quad (r = 0, \dots, k-1) \implies \text{Merge}_k(\mathbf{ct}_{n,0}, \dots, \mathbf{ct}_{n,k-1}) \models_N \sum_{r=0}^{k-1} X^r P_r(X^k)$$

- 实现层面：通过 ring switching + key switching 完成
- 不是公开系数抽取！是对密文的代数操作

系数拆分不保 Slot ($k = 2$ 演示)

设 $P(X) = A(X^2) + XB(X^2)$ 。

对 $\alpha \in \Omega_N$, $\theta = \alpha^2 \in \Omega_n$:

$$P(\alpha) = A(\theta) + \alpha B(\theta) \quad P(-\alpha) = A(\theta) - \alpha B(\theta)$$

系数拆分不保 Slot ($k = 2$ 演示)

设 $P(X) = A(X^2) + XB(X^2)$ 。

对 $\alpha \in \Omega_N$, $\theta = \alpha^2 \in \Omega_n$:

$$P(\alpha) = A(\theta) + \alpha B(\theta) \quad P(-\alpha) = A(\theta) - \alpha B(\theta)$$

$$A(\theta) = \frac{P(\alpha) + P(-\alpha)}{2} \quad \leftarrow \text{线性和与差}$$

$$B(\theta) = \frac{P(\alpha) - P(-\alpha)}{2\alpha} \quad \leftarrow \text{不是 slot 选择!}$$

系数拆分不保 Slot ($k = 2$ 演示)

设 $P(X) = A(X^2) + XB(X^2)$ 。

对 $\alpha \in \Omega_N$, $\theta = \alpha^2 \in \Omega_n$:

$$P(\alpha) = A(\theta) + \alpha B(\theta) \quad P(-\alpha) = A(\theta) - \alpha B(\theta)$$

$$A(\theta) = \frac{P(\alpha) + P(-\alpha)}{2} \quad \leftarrow \text{线性和与差}$$

$$B(\theta) = \frac{P(\alpha) - P(-\alpha)}{2\alpha} \quad \leftarrow \text{不是 slot 选择!}$$

一般情况

纤维 $\pi^{-1}(\theta) = \{\alpha : \alpha^k = \theta\}$ 大小为 k , 内部是 Vandermonde 基变换:

$$P(\alpha) = \sum_{r=0}^{k-1} \alpha^r P_r(\theta), \quad \forall \alpha \in \pi^{-1}(\theta)$$

如果硬要恢复 Slot: B_k 变换

B_k 的定义 (逐纤维)

$$(B_k(u_0, \dots, u_{k-1}))_t(\theta) = \sum_{r=0}^{k-1} \alpha_t(\theta)^r u_r(\theta)$$

其中 $\pi^{-1}(\theta) = \{\alpha_0(\theta), \dots, \alpha_{k-1}(\theta)\}$ 。

如果硬要恢复 Slot: B_k 变换

B_k 的定义 (逐纤维)

$$(B_k(u_0, \dots, u_{k-1}))_t(\theta) = \sum_{r=0}^{k-1} \alpha_t(\theta)^r u_r(\theta)$$

其中 $\pi^{-1}(\theta) = \{\alpha_0(\theta), \dots, \alpha_{k-1}(\theta)\}$ 。

- 若 $u_r(\theta) = P_r(\theta)$, 则 $(B_k u)_t(\theta) = P(\alpha_t(\theta))$ ——恢复了 big-ring slots
- $k = 2$ 时: $Q^\pm(Y) = A(Y) \pm h(Y)B(Y)$, 其中 h 是分支依赖的
- B_k 是 k 个叶子 ciphertext 之间的线性密文变换——代价大

如果硬要恢复 Slot: B_k 变换

B_k 的定义 (逐纤维)

$$(B_k(u_0, \dots, u_{k-1}))_t(\theta) = \sum_{r=0}^{k-1} \alpha_t(\theta)^r u_r(\theta)$$

其中 $\pi^{-1}(\theta) = \{\alpha_0(\theta), \dots, \alpha_{k-1}(\theta)\}$ 。

- 若 $u_r(\theta) = P_r(\theta)$, 则 $(B_k u)_t(\theta) = P(\alpha_t(\theta))$ ——恢复了 big-ring slots
- $k = 2$ 时: $Q^\pm(Y) = A(Y) \pm h(Y)B(Y)$, 其中 h 是分支依赖的
- B_k 是 k 个叶子 ciphertext 之间的线性密文变换——代价大

本文的核心观察

自举核心不需要 B_k ! 因为 EvalMod 需要的是 C2S 之后的 coefficient-packed slots, 而这些坐标正好按余类分组分解。

关键等式：C2S 输出与余类分组的交换

核心不变式

$$\text{coeff}_n(P_r(Y)) = (a_r, a_{r+k}, \dots, a_{r+(n-1)k})$$

$$(\text{coeff}_n(P_0), \dots, \text{coeff}_n(P_{k-1})) = \Pi_k \text{coeff}_N(P(X))$$

关键等式：C2S 输出与余类分组的交换

核心不变式

$$\text{coeff}_n(P_r(Y)) = (a_r, a_{r+k}, \dots, a_{r+(n-1)k})$$

$$(\text{coeff}_n(P_0), \dots, \text{coeff}_n(P_{k-1})) = \Pi_k \text{coeff}_N(P(X))$$

算子等式

$$\left(\bigoplus_{r=0}^{k-1} \text{C2S}_n \right) \circ \text{Split}_k = \Pi_k \circ \text{C2S}_N$$

这就是 $\text{no-}B_k$ 路线的代数根基。
系数拆分的 C2S 输出 = 大环 C2S 输出的余类分组。

核心因式分解定理

定义 (Sandwich 结构)

$$F_N = L_N^{(2)} \circ \Phi_N \circ L_N^{(1)} \quad (\text{大环自举核心})$$

$$F_n = L_n^{(2)} \circ \Phi_n \circ L_n^{(1)} \quad (\text{叶子环自举核心})$$

抽象框架: Split-Compatible Sandwich

定义 (Sandwich 结构)

$$F_N = L_N^{(2)} \circ \Phi_N \circ L_N^{(1)} \quad (\text{大环自举核心})$$

$$F_n = L_n^{(2)} \circ \Phi_n \circ L_n^{(1)} \quad (\text{叶子环自举核心})$$

定理 (定理 4.1: Split-Compatible Sandwich)

若存在 Split, Merge, Π 满足三个兼容条件 (1)(2)(3), 则:

$$F_N = \text{Merge} \circ \left(\bigoplus_{r=0}^{k-1} F_n \right) \circ \text{Split}$$

条件:

- (1) $(\bigoplus_r L_n^{(1)}) \circ \text{Split} = \Pi \circ L_N^{(1)}$
- (2) $(\bigoplus_r \Phi_n) \circ \Pi = \Pi \circ \Phi_N$
- (3) $\text{Merge} \circ (\bigoplus_r L_n^{(2)}) \circ \Pi = L_N^{(2)}$

定理 4.1 的证明

$$\begin{aligned} & \text{Merge} \circ \left(\bigoplus_r F_n \right) \circ \text{Split} \\ &= \text{Merge} \circ \left(\bigoplus_r L_n^{(2)} \circ \Phi_n \circ L_n^{(1)} \right) \circ \text{Split} \\ &= \text{Merge} \circ \left(\bigoplus_r L_n^{(2)} \right) \circ \left(\bigoplus_r \Phi_n \right) \circ \left(\bigoplus_r L_n^{(1)} \right) \circ \text{Split} \end{aligned}$$

定理 4.1 的证明

$$\begin{aligned}
 & \text{Merge} \circ \left(\bigoplus_r F_n \right) \circ \text{Split} \\
 &= \text{Merge} \circ \left(\bigoplus_r L_n^{(2)} \circ \Phi_n \circ L_n^{(1)} \right) \circ \text{Split} \\
 &= \text{Merge} \circ \left(\bigoplus_r L_n^{(2)} \right) \circ \left(\bigoplus_r \Phi_n \right) \circ \left(\bigoplus_r L_n^{(1)} \right) \circ \text{Split} \\
 &\stackrel{(1)}{=} \text{Merge} \circ \left(\bigoplus_r L_n^{(2)} \right) \circ \left(\bigoplus_r \Phi_n \right) \circ \Pi \circ L_N^{(1)} \\
 &\stackrel{(2)}{=} \text{Merge} \circ \left(\bigoplus_r L_n^{(2)} \right) \circ \Pi \circ \Phi_N \circ L_N^{(1)} \\
 &\stackrel{(3)}{=} L^{(2)} \circ \Phi_N \circ L^{(1)} = F_N
 \end{aligned}$$

引理 4.2: EvalMod 与余类分组交换

引理

设 $\Phi_N = f^{\oplus N}$, $\Phi_n = f^{\oplus n}$ (同一标量函数 f), 则:

$$\left(\bigoplus_{r=0}^{k-1} \Phi_n \right) \circ \Pi_k = \Pi_k \circ \Phi_N$$

引理 4.2: EvalMod 与余类分组交换

引理

设 $\Phi_N = f^{\oplus N}$, $\Phi_n = f^{\oplus n}$ (同一标量函数 f), 则:

$$\left(\bigoplus_{r=0}^{k-1} \Phi_n \right) \circ \Pi_k = \Pi_k \circ \Phi_N$$

证明概要.

令 $x = (x_0, \dots, x_{N-1})$.

- 先分组再应用 f : 叶子 r 位置 j 得到 $f(x_{r+jk})$
- 先应用 f 再分组: $f^{\oplus N}(x)$ 的位置 $r + jk$ 是 $f(x_{r+jk})$, 分组后叶子 r 位置 j 也是 $f(x_{r+jk})$

两个路径在每个坐标上都一致。□

关键洞察

这个引理 不要求 f 是线性、加法或乘法的。
它只要求余类分组 Π_k 不含未归一化的坐标依赖缩放。

引理 (引理 4.3: Split-C2S)

$$\left(\bigoplus_{r=0}^{k-1} \text{C2S}_n \right) \circ \text{Split}_k = \Pi_k \circ \text{C2S}_N$$

CKKS 实例化：三个兼容性引理

引理 (引理 4.3: Split-C2S)

$$\left(\bigoplus_{r=0}^{k-1} \text{C2S}_n \right) \circ \text{Split}_k = \Pi_k \circ \text{C2S}_N$$

引理 (引理 4.4: EvalMod 兼容性)

若大环和叶子环使用同一标量 EvalMod 近似 f ，则：

$$\left(\bigoplus_{r=0}^{k-1} \text{EvalMod}_n \right) \circ \Pi_k = \Pi_k \circ \text{EvalMod}_N$$

CKKS 实例化：三个兼容性引理

引理 (引理 4.3: Split-C2S)

$$\left(\bigoplus_{r=0}^{k-1} \text{C2S}_n \right) \circ \text{Split}_k = \Pi_k \circ \text{C2S}_N$$

引理 (引理 4.4: EvalMod 兼容性)

若大环和叶子环使用同一标量 EvalMod 近似 f , 则:

$$\left(\bigoplus_{r=0}^{k-1} \text{EvalMod}_n \right) \circ \Pi_k = \Pi_k \circ \text{EvalMod}_N$$

引理 (引理 4.5: S2C-Merge 兼容性)

$$\text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} \text{S2C}_n \right) \circ \Pi_k = \text{S2C}_N$$

主定理 4.6: CKKS 核心因式分解

定理 (定理 4.6)

在理想归一化消息层模型下:

$$\text{S2C}_N \circ \text{EvalMod}_N \circ \text{C2S}_N = \text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} \text{S2C}_n \circ \text{EvalMod}_n \circ \text{C2S}_n \right) \circ \text{Split}_k$$

主定理 4.6: CKKS 核心因式分解

定理 (定理 4.6)

在理想归一化消息层模型下:

$$\text{S2C}_N \circ \text{EvalMod}_N \circ \text{C2S}_N = \text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} \text{S2C}_n \circ \text{EvalMod}_n \circ \text{C2S}_n \right) \circ \text{Split}_k$$

证明思路.

定理 4.1 中取:

$$L^{(1)} = \text{C2S}, \quad \Phi = \text{EvalMod}, \quad L^{(2)} = \text{S2C}$$

三条件恰为引理 4.3、4.4、4.5。代入即得。□

一个理想层的等式——实现需要另行控制误差

加入预处理：完整因式分解

预处理定义

$\text{Pre}_d = \text{ModRaise}_d \circ \text{ScaleDown}_d$

ScaleDown_d 的作用：公开归一化， $\mathbf{ct} \models \Delta P \implies \text{ScaleDown}(\mathbf{ct}) \models \Delta' \rho P$

ModRaise_d 的作用：模数提升（理想层不变消息）

加入预处理：完整因式分解

预处理定义

$\text{Pre}_d = \text{ModRaise}_d \circ \text{ScaleDown}_d$

ScaleDown_d 的作用：公开归一化, $\mathbf{ct} \models \Delta P \implies \text{ScaleDown}(\mathbf{ct}) \models \Delta' \rho P$

ModRaise_d 的作用：模数提升（理想层不变消息）

预处理与拆分的兼容性 (Eq 4)

$$\left(\bigoplus_{r=0}^{k-1} \text{Pre}_n \right) \circ \text{Split}_k = \text{Split}_k \circ \text{Pre}_N$$

因为 $\rho P(X) = \sum_r X^r (\rho P_r)(X^k)$ （同一缩放因子 ρ ）。

加入预处理：完整因式分解

预处理定义

$\text{Pre}_d = \text{ModRaise}_d \circ \text{ScaleDown}_d$

ScaleDown_d 的作用：公开归一化， $\mathbf{ct} \models \Delta P \implies \text{ScaleDown}(\mathbf{ct}) \models \Delta' \rho P$

ModRaise_d 的作用：模数提升（理想层不变消息）

预处理与拆分的兼容性 (Eq 4)

$$\left(\bigoplus_{r=0}^{k-1} \text{Pre}_n \right) \circ \text{Split}_k = \text{Split}_k \circ \text{Pre}_N$$

因为 $\rho P(X) = \sum_r X^r (\rho P_r)(X^k)$ （同一缩放因子 ρ ）。

定义完整自举映射 $\text{BTS}_d = \text{S2C}_d \circ \text{EvalMod}_d \circ \text{C2S}_d \circ \text{Pre}_d$ ，则：

$$\text{BTS}_N = \text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} \text{BTS}_n \right) \circ \text{Split}_k \quad (\text{Eq 5})$$

三条实现路线的语义对比

推荐: **Split-first** 全叶子预处理

$$\text{Split} \rightarrow \bigoplus_r \text{ScaleDown}_n \rightarrow \bigoplus_r \text{ModRaise}_n \rightarrow \bigoplus_r \text{C2S}_n \rightarrow \bigoplus_r \text{EvalMod}_n \rightarrow \bigoplus_r \text{S2C}_n \rightarrow \text{Merge}$$

ScaleDown_n 和 ModRaise_n 作为配对预处理放在叶子环内。实验表明精度显著优于混合路线。

三条实现路线的语义对比

推荐：Split-first 全叶子预处理

$$\text{Split} \rightarrow \bigoplus_r \text{ScaleDown}_n \rightarrow \bigoplus_r \text{ModRaise}_n \rightarrow \bigoplus_r \text{C2S}_n \rightarrow \bigoplus_r \text{EvalMod}_n \rightarrow \bigoplus_r \text{S2C}_n \rightarrow \text{Merge}$$

ScaleDown_n 和 ModRaise_n 作为配对预处理放在叶子环内。实验表明精度显著优于混合路线。

对比：旧路线 (Top 预处理)

$$\text{ScaleDown}_N \rightarrow \text{ModRaise}_N \rightarrow \text{Split} \rightarrow \bigoplus_r \text{C2S}_n \rightarrow \dots$$

预处理开销留在大环，违背分解目的。

对比：混合路线 (仅 ModRaise 移到叶子)

$$\text{ScaleDown}_N \rightarrow \text{Split} \rightarrow \bigoplus_r \text{ModRaise}_n \rightarrow \dots$$

输出精度崩溃 (0.96 bits)——Leaf EvalMod 的归一化预期被破坏!

实现与实验

Go 接口定义

```
type leafBootstrapper interface {  
    ScaleDown(*rlwe.Ciphertext) (*rlwe.Ciphertext, error)  
    ModUp(*rlwe.Ciphertext) (*rlwe.Ciphertext, error)  
    CoeffsToSlots(*rlwe.Ciphertext) (*rlwe.Ciphertext, error)  
    EvalMod(*rlwe.Ciphertext) (*rlwe.Ciphertext, error)  
    SlotsToCoeffs(*rlwe.Ciphertext) (*rlwe.Ciphertext, error)  
    MaxBootLevel() int  
}
```

实现架构：leafBootstrapper 接口

Go 接口定义

```
type leafBootstrapper interface {  
    ScaleDown(*rlwe.Ciphertext) (*rlwe.Ciphertext, error)  
    ModUp(*rlwe.Ciphertext) (*rlwe.Ciphertext, error)  
    CoeffsToSlots(*rlwe.Ciphertext) (*rlwe.Ciphertext, error)  
    EvalMod(*rlwe.Ciphertext) (*rlwe.Ciphertext, error)  
    SlotsToCoeffs(*rlwe.Ciphertext) (*rlwe.Ciphertext, error)  
    MaxBootLevel() int  
}
```

- 当前实现：lattigoLeafBootstrapper (标准 Lattigo CKKS Bootstrapping)
- 新增实现：anyuLeafBootstrapper (subKey Bootstrapping)
- 叶子引擎完全可替换——定理不绑定具体 Bootstrapping 实现

目标 Workload: 容量驱动而非深度驱动

适用条件

- ① 容量条件: $S(n) < M \leq S(N)$, $N = kn$
- ② 深度条件: $D_{\text{cycle}} \leq L(Q_{\text{leaf}})$

大环因 **packing** 容量而选择，不是因深度。

目标 Workload: 容量驱动而非深度驱动

适用条件

- ① 容量条件: $S(n) < M \leq S(N)$, $N = kn$
- ② 深度条件: $D_{\text{cycle}} \leq L(Q_{\text{leaf}})$

大环因 **packing** 容量而选择, 不是因深度。

典型场景

- 批量 SIMD 推理: $M = B \cdot F$ (批量大小 $\times$ 特征数)
- 数据库聚合: $M = R \cdot a$ (记录数 $\times$ 每记录 slots)
- 大向量分析: $M = G \cdot p$ (独立任务数 $\times$ 每任务维度)

目标 Workload: 容量驱动而非深度驱动

适用条件

- ① 容量条件: $S(n) < M \leq S(N)$, $N = kn$
- ② 深度条件: $D_{\text{cycle}} \leq L(Q_{\text{leaf}})$

大环因 **packing** 容量而选择, 不是因深度。

典型场景

- 批量 SIMD 推理: $M = B \cdot F$ (批量大小 $\times$ 特征数)
- 数据库聚合: $M = R \cdot a$ (记录数 $\times$ 每记录 slots)
- 大向量分析: $M = G \cdot p$ (独立任务数 $\times$ 每任务维度)

什么时候不适合?

深度驱动的 workload ($D_{\text{cycle}} > L(Q_{\text{leaf}})$), 降低模数会导致自举频率增加抵消每自举节省的时间。

方案安全 = 最弱 RLWE 实例

$$\lambda_{\text{scheme}} = \min\{\lambda(N, Q_{\text{top}}, P_{\text{top}}), \lambda(n, Q_{\text{leaf}}, P_{\text{leaf}}), \lambda_{\text{rs/ks}}\}$$

方案安全 = 最弱 RLWE 实例

$$\lambda_{\text{scheme}} = \min\{\lambda(N, Q_{\text{top}}, P_{\text{top}}), \lambda(n, Q_{\text{leaf}}, P_{\text{leaf}}), \lambda_{\text{rs/ks}}\}$$

关键点

拆分后叶子密文和叶子密钥暴露在维度 n 下。

安全瓶颈在叶子环，不在大环！大环的安全余量不能掩盖不安全的叶子。

方案安全 = 最弱 RLWE 实例

$$\lambda_{\text{scheme}} = \min\{\lambda(N, Q_{\text{top}}, P_{\text{top}}), \lambda(n, Q_{\text{leaf}}, P_{\text{leaf}}), \lambda_{\text{rs/ks}}\}$$

关键点

拆分后叶子密文和叶子密钥暴露在维度 n 下。

安全瓶颈在叶子环，不在大环！大环的安全余量不能掩盖不安全的叶子。

128-bit 安全参考表（工作估值）

$\log N$	$\log(PQ)$ 参考上限
15 ($N = 2^{15}$)	≈ 868
16 ($N = 2^{16}$)	≈ 1747
17 ($N = 2^{17}$)	≈ 3523

总误差分解

$$e_{\text{out}} = e_{\text{enc}} + e_{\text{split}} + e_{\text{sd}} + e_{\text{mr}} + e_{\text{C2S}} + e_{\text{EvalMod}} + e_{\text{S2C}} + e_{\text{merge}}$$

总误差分解

$$e_{\text{out}} = e_{\text{enc}} + e_{\text{split}} + e_{\text{sd}} + e_{\text{mr}} + e_{\text{C2S}} + e_{\text{EvalMod}} + e_{\text{S2C}} + e_{\text{merge}}$$

正确性条件

$$\|e_{\text{out}}\| \ll \Delta_{\text{out}}, \quad \|\Delta m + e\| < Q/2$$

总误差分解

$$e_{\text{out}} = e_{\text{enc}} + e_{\text{split}} + e_{\text{sd}} + e_{\text{mr}} + e_{\text{C2S}} + e_{\text{EvalMod}} + e_{\text{S2C}} + e_{\text{merge}}$$

正确性条件

$$\|e_{\text{out}}\| \ll \Delta_{\text{out}}, \quad \|\Delta m + e\| < Q/2$$

精度公式

$$E \approx 2^{-p} \quad (p \text{ 为 L1 precision in bits})$$

- 安全要求拉低 Q —— 增加 RLWE 安全性
- 正确性要求拉高 Q —— 减少模数溢出风险
- 安全与精度是双向拉锯

① **Galois Key 缺失**：subKey 的 CtS 需要特殊旋转密钥

⇒ 生成 subring CtS 专用 Galois keys，挂到 evaluator

实验历程：subKey 集成三步走

- ① **Galois Key 缺失**：subKey 的 CtS 需要特殊旋转密钥

⇒ 生成 subring CtS 专用 Galois keys，挂到 evaluator

- ② **精度崩溃** → 负数 **bits**：CtS 第一段折叠深度不对

$$\text{leaf CtS depth} = \frac{2^{\text{leafLogN}}}{2^{\text{subringLogN}}} \begin{cases} 2^{15}/2^{12} = 2^3 = 3 & (\text{top} = 16) \\ 2^{16}/2^{12} = 2^4 = 4 & (\text{top} = 17) \end{cases}$$

⇒ 加入 ManualDepthSplit 自动修正

实验历程：subKey 集成三步走

- ① **Galois Key 缺失**：subKey 的 CtS 需要特殊旋转密钥

⇒ 生成 subring CtS 专用 Galois keys，挂到 evaluator

- ② **精度崩溃 → 负数 bits**：CtS 第一段折叠深度不对

$$\text{leaf CtS depth} = \frac{2^{\text{leafLogN}}}{2^{\text{subringLogN}}} \begin{cases} 2^{15}/2^{12} = 2^3 = 3 & (\text{top} = 16) \\ 2^{16}/2^{12} = 2^4 = 4 & (\text{top} = 17) \end{cases}$$

⇒ 加入 ManualDepthSplit 自动修正

- ③ **模数链对齐**：leaf Q/P basis $\equiv$ top Q/P basis (exact-basis 设计)

⇒ split/merge 与 leaf Bootstrapping 的 modulus 完全兼容

配置	$\log Q$	$\log QP$	精度	时间	判定
default lowq	868	929	16.16	6.73s	基准
Eval56 CtS46	866	927	16.42	6.55s	↑
Eval56 StC26 CtS50	866	927	16.74	6.59s	最优
Eval57 CtS43	867	928	13.77	6.69s	CtS 太低
Eval60 CtS33	867	928	3.77	6.62s	明显失败
StC23 CtS50	867	928	14.02	6.02s	StC 太低
subringLogN=13	868	929	-5.52	8.03s	失败

配置	$\log Q$	$\log QP$	精度	时间	判定
default lowq	868	929	16.16	6.73s	基准
Eval56 CtS46	866	927	16.42	6.55s	↑
Eval56 StC26 CtS50	866	927	16.74	6.59s	最优
Eval57 CtS43	867	928	13.77	6.69s	CtS 太低
Eval60 CtS33	867	928	3.77	6.62s	明显失败
StC23 CtS50	867	928	14.02	6.02s	StC 太低
subringLogN=13	868	929	-5.52	8.03s	失败

- CtS 不能压太低 (< 46 即恶化)
- StC 可以从 30 降到 26, 但不能到 23
- 安全裕量: $\log QP = 927$ vs 参考 868 → 约 -59 bits
- 结论: 工程可行, 安全叙事不干净, 不适合主推

配置	$\log Q$	$\log QP$	层数	精度	时间	安全
lowq tuned	866	927	1	15.47	45.7s	+820
lowq default	868	929	1	14.64	43.0s	+818
i1 default	1425	1730	15	19.40	52.0s	+17
i1 CtS=55	1434	1739	15	19.42	50.5s	+8
i1 d255	1415	1720	13	19.39	45.3s	+27

配置	$\log Q$	$\log QP$	层数	精度	时间	安全
lowq tuned	866	927	1	15.47	45.7s	+820
lowq default	868	929	1	14.64	43.0s	+818
i1 default	1425	1730	15	19.40	52.0s	+17
i1 CtS=55	1434	1739	15	19.42	50.5s	+8
i1 d255	1415	1720	13	19.39	45.3s	+27

主推：i1 default

top $\log N = 17$, **leaf** $\log N = 16$, **layers=1**
 $\log Q = 1425$, $\log QP = 1730$ (参考 1747, 裕量 +17)

- 叶子正好是 $2^{16} \rightarrow 2^{12}$ subKey 设置 (接近 Wang 2025 原文设置)
- CtS=55 只提升 0.02 bits 但吃掉 9 bits 安全裕量 → 不值得
- Mod1Degree=255 没提升精度且减少 2 层输出

完整对照实验：四种方法 (I1)

方法	拆分	subKey	$\log QP$	安全	精度	时间
组合方案	是	是	1730	+17	19.40	52.0s
只有拆分	是	否	1696	+51	20.47	63.2s
只有 subKey	否	是	1730	+1793	18.61	135.1s
直接 baseline	否	否	1696	+1827	20.87	143.3s

完整对照实验：四种方法 (I1)

方法	拆分	subKey	$\log QP$	安全	精度	时间
组合方案	是	是	1730	+17	19.40	52.0s
只有拆分	是	否	1696	+51	20.47	63.2s
只有 subKey	否	是	1730	+1793	18.61	135.1s
直接 baseline	否	否	1696	+1827	20.87	143.3s

加速比

- vs 直接 baseline: $143.3/52.0 \approx 2.75\times$
 - vs 只有 subKey: $135.1/52.0 \approx 2.60\times$
 - vs 只有拆分: $63.2/52.0 \approx 1.21\times$
-
- 组合方案 = 外部分解加速 + 内部 subKey 加速 (叠加!)
 - subKey 替换叶子普通 Bootstrapping 带来 $1.21\times$ 额外加速, 牺牲约 1 bit 精度

#	方法	分类	$\log Q$	$\log QP$	安全	层数	精度	时间
1	组合 · i1	主推	1425	1730	+17	15	19.40	52.0s
2	组合 · CtS55	对比	1434	1739	+8	15	19.42	50.5s
3	组合 · d255	对比	1415	1720	+27	13	19.39	45.3s
4	只拆分	对照	1391	1696	+51	15	20.47	63.2s
5	只 subKey	对照	1425	1730	+1793	15	18.61	135.1s
6	直接 baseline	基线	1391	1696	+1827	15	20.87	143.3s

方法	$\log QP$	安全	精度	时间	备注
组合方案	927	-59	16.74	6.59s	工程可行 条件不满足
只有拆分	960	-92	失败	—	
只有 subKey	927	+820	15.92	32.9s	
直接 baseline	960	+787	21.97	60.9s	

方法	$\log QP$	安全	精度	时间	备注
组合方案	927	-59	16.74	6.59s	工程可行 条件不满足
只有拆分	960	-92	失败	-	
只有 subKey	927	+820	15.92	32.9s	
直接 baseline	960	+787	21.97	60.9s	

top=16 的教训

- 组合方案加速比惊人：vs baseline $\approx 9.2\times$ ，vs subKey $\approx 5.0\times$
- 但 leaf15 安全裕量为负 ($\log QP = 927 > 868$)
- only-split 路线在 top=16 下不稳定 (split-first 失败，旧路线归一化不对)
- 结论：适合作为工程验证，不适合作为论文安全参数

特殊模数布局

$$q_0 \parallel Q_{\text{circuit}} \parallel Q_{\text{StC}} \parallel Q_{\text{EvalMod}} \parallel Q_{\text{CtS}}$$

特殊模数布局

$$q_0 \parallel Q_{\text{circuit}} \parallel Q_{\text{StC}} \parallel Q_{\text{EvalMod}} \parallel Q_{\text{CtS}}$$

参数	默认值 (l1)
$\log(\text{default scale})$	35
q_0	[45]
Q_{StC}	[35, 35, 35] (3 primes)
Q_{EvalMod}	EvalModLevels $\times$ [60]
Q_{CtS}	[60, 60, 60, 60] (4 primes)
$P_{\text{bootstrap}}$	[61, 61, 61, 61, 61] (5 primes)
Mod1Degree	127
DoubleAngle	3

$$\text{EvalModLevels} = \lceil \log_2(\text{Mod1Degree}) \rceil + \text{DoubleAngle}$$

总结与展望

主推参数

top $\log N = 17$, leaf $\log N = 16$, layers = 1, l1 profile

指标	数值
安全参考 ($\log QP$)	1747
实际 $\log QP$	1730
安全裕量	+17 bits
输出计算层	15
平均 L1 精度	19.40 bits
运行时间	52.0s
vs 直接 baseline 加速	2.75×
vs 只有 subKey 加速	2.60×
vs 只有拆分加速	1.21×

方法边界：什么时候用？

适用场景

- 大环因 **packing** 容量而选择 ($S(n) < M \leq S(N)$)
- 刷新周期深度在叶子安全模数预算内 ($D_{\text{cycle}} \leq L(Q_{\text{leaf}})$)
- 需要保持单一大环密文布局

方法边界：什么时候用？

适用场景

- 大环因 **packing** 容量而选择 ($S(n) < M \leq S(N)$)
- 刷新周期深度在叶子安全模数预算内 ($D_{\text{cycle}} \leq L(Q_{\text{leaf}})$)
- 需要保持单一大环密文布局

不适用场景

- 深度驱动的 workload（降低模数会增加自举频率）
- 没有容量需求，多个小环密文自然够用
- $B(Q_{\text{leaf}}) \cdot T_{\text{boot}}^{\text{split}} > B(Q_{\text{top}}) \cdot T_{\text{boot}}^{\text{direct}}$

$$B(Q_{\text{leaf}}) \cdot T_{\text{boot}}^{\text{split}} < B(Q_{\text{top}}) \cdot T_{\text{boot}}^{\text{direct}} \quad (\text{收益条件})$$

- ① 重复实验：对第一名参数多次运行，统计均值与方差
- ② **Stage trace**：定位剩余约 2 bits 精度损失的具体来源

$$e_{\text{out}} = e_{\text{enc}} + e_{\text{split}} + e_{\text{sd}} + e_{\text{mr}} + e_{\text{C2S}} + e_{\text{EvalMod}} + e_{\text{S2C}} + e_{\text{merge}}$$

- ③ 内存统计：增加 evaluation key size 和内存占用报告
- ④ 安全核算：用正式 LWE estimator 替换当前的参考值
- ⑤ 论文定稿：将实验数据写入 Section 5 的最终版本

- ① 语义洞察：CKKS 自举核心需要的是 C2S 后的 coefficient-packed slots，不是原始 canonical slots。系数拆分恰好保持了 coefficient-packed 坐标的余类分组结构。

- ① 语义洞察：CKKS 自举核心需要的是 C2S 后的 coefficient-packed slots，不是原始 canonical slots。系数拆分恰好保持了 coefficient-packed 坐标的余类分组结构。
- ② 形式化证明：建立了抽象 Sandwich 因式分解框架 (Theorem 4.1)，并在 CKKS 上证明了完整因式分解 (Theorem 4.6)，不需要 B_k 槽恢复变换。

- ① 语义洞察：CKKS 自举核心需要的是 C2S 后的 coefficient-packed slots，不是原始 canonical slots。系数拆分恰好保持了 coefficient-packed 坐标的余类分组结构。
- ② 形式化证明：建立了抽象 Sandwich 因式分解框架 (Theorem 4.1)，并在 CKKS 上证明了完整因式分解 (Theorem 4.6)，不需要 B_k 槽恢复变换。
- ③ 工程实现：实现了 leafBootstrapper 接口，验证了 split-first 全叶子预处理路线的正确性。

- ① 语义洞察：CKKS 自举核心需要的是 C2S 后的 coefficient-packed slots，不是原始 canonical slots。系数拆分恰好保持了 coefficient-packed 坐标的余类分组结构。
- ② 形式化证明：建立了抽象 Sandwich 因式分解框架 (Theorem 4.1)，并在 CKKS 上证明了完整因式分解 (Theorem 4.6)，不需要 B_k 槽恢复变换。
- ③ 工程实现：实现了 leafBootstrapper 接口，验证了 split-first 全叶子预处理路线的正确性。
- ④ 组合加速：成功将 no- B_k 外部分解与 subKey (AnyuWang 2025) 内层自举叠加，本地实验获得约 $2.75\times$ 总加速比。

谢谢！

欢迎提问

- 代码：ckks-subring-compression/5-7/
- 论文草案：Section2-4_ 整合版草案.md
- 实验报告：最新 _LEAF_EXPERIMENT_REPORT.md

附录：完整公式参考 I

- ① 环: $\mathcal{R}_d = \mathbb{C}[X]/(X^d + 1)$, $\mathcal{R}_{q,d} = \mathbb{Z}_q[X]/(X^d + 1)$
- ② 语义: $\mathbf{ct} \models_{d,\Delta,s} P(X)$
- ③ 典范嵌入: $\text{can}_d : \mathcal{R}_d \rightarrow \text{Can}_d$
- ④ 系数向量: $\text{coeff}_d(P) = (a_0, \dots, a_{d-1})$
- ⑤ C2S: $\text{C2S}_d = \text{coeff}_d \circ \text{can}_d^{-1}$
- ⑥ S2C: $\text{S2C}_d = \text{can}_d \circ \text{coeff}_d^{-1}$
- ⑦ EvalMod: $\text{EvalMod}_d = f^{\oplus d}$
- ⑧ 子环分解: $P(X) = \sum_{r=0}^{k-1} X^r P_r(X^k)$
- ⑨ 余类分组: $\Pi_k p = (S_0 p, \dots, S_{k-1} p)$
- ⑩ CoeffSplit: $\text{CoeffSplit}_k(P) = (P_0, \dots, P_{k-1})$
- ⑪ Split 语义: $\mathbf{ct}_N \mapsto (\mathbf{ct}_{n,0}, \dots, \mathbf{ct}_{n,k-1})$
- ⑫ 纤维: $\pi(\alpha) = \alpha^k$, $\pi^{-1}(\theta) = \{\alpha : \alpha^k = \theta\}$
- ⑬ 纤维内关系: $P(\alpha) = \sum_{r=0}^{k-1} \alpha^r P_r(\theta)$

附录：完整公式参考 II

- 14 $B_k: (B_k u)_t(\theta) = \sum_{r=0}^{k-1} \alpha_t(\theta)^r u_r(\theta)$
- 15 核心不变式: $(\text{coeff}_n(P_0), \dots) = \Pi_k \text{coeff}_N(P)$
- 16 算子等式: $(\bigoplus_r \text{C2S}_n) \circ \text{Split}_k = \Pi_k \circ \text{C2S}_N$
- 17 Thm 4.1: $F_N = \text{Merge} \circ (\bigoplus_r F_n) \circ \text{Split}$
- 18 条件 (1): $(\bigoplus_r L_n^{(1)}) \circ \text{Split} = \Pi \circ L_N^{(1)}$
- 19 条件 (2): $(\bigoplus_r \Phi_n) \circ \Pi = \Pi \circ \Phi_N$
- 20 条件 (3): $\text{Merge} \circ (\bigoplus_r L_n^{(2)}) \circ \Pi = L_N^{(2)}$
- 21 Lemma 4.2: $(\bigoplus_r f^{\oplus n}) \circ \Pi_k = \Pi_k \circ f^{\oplus N}$
- 22 Lemma 4.3: $(\bigoplus_r \text{C2S}_n) \circ \text{Split}_k = \Pi_k \circ \text{C2S}_N$
- 23 Lemma 4.4: $(\bigoplus_r \text{EvalMod}_n) \circ \Pi_k = \Pi_k \circ \text{EvalMod}_N$
- 24 Lemma 4.5: $\text{Merge}_k \circ (\bigoplus_r \text{S2C}_n) \circ \Pi_k = \text{S2C}_N$
- 25 Thm 4.6: $\text{S2C}_N \circ \text{EvalMod}_N \circ \text{C2S}_N = \text{Merge}_k \circ (\bigoplus_r \text{S2C}_n \circ \text{EvalMod}_n \circ \text{C2S}_n) \circ \text{Split}_k$
- 26 $\text{Pre}_d = \text{ModRaise}_d \circ \text{ScaleDown}_d$

附录：完整公式参考 III

- 27 Eq(4): $(\bigoplus_r \text{Pre}_n) \circ \text{Split}_k = \text{Split}_k \circ \text{Pre}_N$
- 28 $\text{BTS}_d = \text{S2C}_d \circ \text{EvalMod}_d \circ \text{C2S}_d \circ \text{Pre}_d$
- 29 Eq(5): $\text{BTS}_N = \text{Merge}_k \circ (\bigoplus_r \text{BTS}_n) \circ \text{Split}_k$
- 30 容量条件: $S(n) < M \leq S(N), N = kn$
- 31 深度条件: $D_{\text{cycle}} \leq L(Q_{\text{leaf}})$
- 32 安全: $\lambda_{\text{scheme}} = \min(\lambda_{\text{top}}, \lambda_{\text{leaf}}, \lambda_{\text{rs/ks}})$
- 33 噪声: $e_{\text{out}} = e_{\text{enc}} + \dots + e_{\text{merge}}$
- 34 代价: $T_{\text{op}} \approx C \cdot d \log d \cdot \#\text{primes}(Q)$
- 35 收益条件: $B(Q_{\text{leaf}}) \cdot T^{\text{split}} < B(Q_{\text{top}}) \cdot T^{\text{direct}}_3$
- 36 CtS 深度: $\text{depth} = 2^{\text{leafLogN}} / 2^{\text{subringLogN}}$
- 37 EvalMod 层数: $\lceil \log_2(\text{Mod1Degree}) \rceil + \text{DoubleAngle}$
- 38 安全参考: $\log N = 15 \rightarrow 868, 16 \rightarrow 1747, 17 \rightarrow 3523$